```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Same kinds of imports as the ones you'd do on c++ but not exactly the same
For printing and reading usr input
Converting txt to num
General text processing

```c
#define MAX_JOBS 10
#define MAX_PARTITIONS 20
```

defining set number so instead of repeating nums just type MAX_JOBS etc. set variable

HAVE TO SET DATA STRUCTS LIKE IN C++
Job structure here -

```c
typedef struct {
    int job_id;
    int size;
    int arrival_time;
    int processing_time;
    int remaining_time;
    int start_address;
    int is_allocated;
    int is_completed;
} Job;
```

ALL is in INT form for simplicity. Names are self explanatory you already know what each variable is for

How do we structure the partitions now??
Partition structure here -

```c
typedef struct {
    int start_address;
    int size;
    int job_id;
    int is_free;
    int remaining_time;
} Partition;
```

this helsp me show where each block of the partition is. Its variable and dynamic remember so we have to make and show that each partition and its size is dynamic. Also need to know if job is free or not and like on mam's whiteboard need the remaining time

Memory managing logic is here –

```c
typedef struct {
    Job jobs[MAX_JOBS];
    Partition partitions[MAX_PARTITIONS];
```

```
    int memory_size;
    int compaction_interval;
    int current_time;
    int partition_count;
    int job_count;
} MemoryManager;

```

Job max job which is 10
Partitions max partitions is 20 FOR NOW.. wasn't able to ask mam the limit but this LOOKS reasonable.
Compaction interval is SELF EXPLANATORY. Name stuff so we all know what that is
Current time means current time interval like how mam did it
Partition count how many sections we got
Job count how many jobs we got

We use this to keep track of EVERYTHING.

INSERT DEFAULT VALUES. (ill think of the editing later.)

```
void init_default_values(MemoryManager *mm) {
    mm->memory_size = 200;
    mm->compaction_interval = 3;
    mm->current_time = 0;
    mm->job_count = 10;
    mm->partition_count = 1;

    // DEFAULT job data
    int sizes[] = {50, 20, 30, 70, 80, 20, 10, 30, 20, 50};
    int arrivals[] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
    int processing[] = {5, 4, 3, 2, 6, 7, 1, 3, 4, 5};
```

HOW do we show it with no damn gui but like how mam did it in the whiteboard??

Set and define the jobsizes arrivals processing in an array

```
void display_memory_state(MemoryManager *mm) {
    // yadda yadda code to draw the boxes ugh
    printf("+");
    for (int j = 0; j < widths[i]; j++) {
        printf("-");
    }
    printf("+\n");

```

I tried manually…

```
+-------+-------+-------+
| J1(5) | J2(4) |       |
+-------+-------+-------+
0       50      70      200

```

Here's how its supposed to look..
In parentheses is the time remaining..
Those labels below change theyre NOT fixed
Now have to find if a free partition is avail.

```
int find_free_partition(MemoryManager *mm, int size) {
    for (int i = 0; i < mm->partition_count; i++) {
        if (mm->partitions[i].is_free && mm->partitions[i].size >= size) {
```

```
            return i;
        }
    }
    return -1;
}

```

Go through EACH section and is it big enough? YES?, return what section that is and if we gone thru everything and nothing return -1 no spot found

Allocate_job func
Using this to allocate jobs

```
void allocate_job(MemoryManager *mm, int job_index, int partition_index) {
    Job *job = &mm->jobs[job_index];

    int original_start = mm->partitions[partition_index].start_address;
    int original_size = mm->partitions[partition_index].size;

    job->start_address = original_start;
    job->is_allocated = 1;

```

Get the job info FIRST. Remember where the partition starts and how big it is tell the job where its going to be IN and mark it as allocated

NOW IF the partition is bigger than what the job needs we have to split it
The job goes to the front of the section and the leftover becomes a free sec.

COMPACTION
Mam was explicit on this and how it works

```
void compact_memory(MemoryManager *mm) {
    printf("\nCompaction Started\n");

    Partition temp_partitions[MAX_PARTITIONS];
    int temp_count = 0;
    int current_addr = 0;

    // should collect partitions
    for (int i = 0; i < mm->partition_count; i++) {
        if (!mm->partitions[i].is_free) {
            temp_partitions[temp_count] = mm->partitions[i];
            temp_partitions[temp_count].start_address = current_addr;
            current_addr += temp_partitions[temp_count].size;
            temp_count++;
        }
    }

```

Go thru ALL the sections find which has a job in it move them start from address 0 move all those jobs right next to each other update the addresses and append that to the respective jobs
Now we have lots of empty space at the end of the memory.

New arrivals:

```
void process_arriving_jobs(MemoryManager *mm) {
    for (int i = 0; i < mm->job_count; i++) {
        Job *job = &mm->jobs[i];

        // new jobs
        if (job->arrival_time == mm->current_time && !job->is_allocated && !job->is_completed) {
            int partition_idx = find_free_partition(mm, job->size);
            if (partition_idx != -1) {
                allocate_job(mm, i, partition_idx);
            } else {
                printf("Job %d arrived but cannot be allocated (insufficient memory)\n", job->job_id);
            }
        }

```

GO THROUGH ALL THE JOBS IN OUR LIST.
Does the job arrival time match the current??
DO NOT do the same job twice and then find a free partition to put our job in OTHERWISE. Do not process job yet.

MAINLOOP

```
void run_complete_simulation(MemoryManager *mm) {
    printf("\nStarting simulation..\n");

    while (!all_jobs_completed(mm) && mm->current_time < MAX_SIMULATION_TIME) {
        simulate_step(mm);
    }
}

```

We KEEP GOING UNTIL ALL JOBS ARE PROCESSED or WE'VE REACHED THE CUTOFF OF 50 time units , 50 is arbitrary and sounds like a reasonable of time units before the output gets too long.

For every time step:

```
void simulate_step(MemoryManager *mm) {
    printf("\nTime %d\n", mm->current_time);

    // compact first if it's time
    if (mm->current_time > 0 && mm->current_time % mm->compaction_interval == 0) {
        compact_memory(mm);
    }

    process_arriving_jobs(mm);
    process_running_jobs(mm);
    display_memory_state(mm);

    mm->current_time++;
}

```

IN ORDER!
Print the current time
Every n time unit run compaction

Process jobs that have just arrivd
Reduce the remaining time for processing jobs REMOVE done jobs
Then we draw the current state shoddily
Move to the next time unit

INPUT VALIDATION (INCOMPLETE)

```c
int is_valid_integer(const char* str) {
    if (str == NULL || *str == '\0') return 0;

    while (isspace(*str)) str++;

    if (*str == '+' || *str == '-') str++;

    // must have at least one digit
    if (!isdigit(*str)) return 0;

```

Is there any input at all? Are there spaces? Atleast one digit?

```c
int get_validated_integer(const char* prompt, int min, int max, int default_value) {
    char input[100];
    int value;

    while (1) {
        printf("%s", prompt);
        if (get_input_line(input, sizeof(input)) && strlen(input) > 0) {
            // check if valid number
            // check if in range
            // return value if good
        } else {
            return default_value;
        }
    }
}

```

Ask for input if there is check if valid integer and whether its in the range if good use the input
If ENTER is pressed with no input , USE def value

POINTERS

```c
Job *job = &mm->jobs[job_index];

```

mm->jobs[job index] to get the address of the job
*job to make a variable to store that address
Job now refers to the job in the memory

We need to do a faux queue since its not built in unlike c++ so we just work with arrays and scan them so manually implement the dynamic memory allocation , pointers , queue , and the pushing and popping

All this:

```
void process_arriving_jobs(MemoryManager *mm) {
    for (int i = 0; i < mm->job_count; i++) {
        Job *job = &mm->jobs[i];

        // new jobs arriving right now
        if (job->arrival_time == mm->current_time && !job->is_allocated && !job->is_completed) {
            int partition_idx = find_free_partition(mm, job->size);
            if (partition_idx != -1) {
                allocate_job(mm, i, partition_idx);
            } else {
                printf("Job %d arrived but cannot be allocated (insufficient memory)\n", job->job_id);
            }
        }

        // RETRY jobs that arrived earlier but couldn't get memory
        if (job->arrival_time < mm->current_time && !job->is_allocated && !job->is_completed) {
            int partition_idx = find_free_partition(mm, job->size);
            if (partition_idx != -1) {
                printf("Job %d now allocated (memory became available)\n", job->job_id);
                allocate_job(mm, i, partition_idx);
            }
        }
    }
}

```

Could’ve been this short in js

```
let waitingJobs = [];
waitingJobs.push(job);
let nextJob = waitingJobs.shift();
```

Now in the default values everything arrived at time 0 so scanning the array from 0 to n implements fcfs

The dynamic partition splitting and merging

```
void allocate_job(MemoryManager *mm, int job_index, int partition_index) {
    // setup

    if (original_size == job->size) {
        // if equal mark occupied
        mm->partitions[partition_index].job_id = job->job_id;
        mm->partitions[partition_index].is_free = 0;
    } else {
        // SHIFT all partitions to the right to make room
        for (int i = mm->partition_count; i > partition_index + 1; i--) {
            mm->partitions[i] = mm->partitions[i - 1];
        }

        // create the partition here
        mm->partitions[partition_index].job_id = job->job_id;
        mm->partitions[partition_index].size = job->size;
        mm->partitions[partition_index].is_free = 0;

        // make new free partition from leftover
        mm->partitions[partition_index + 1].start_address = original_start + job->size;
        mm->partitions[partition_index + 1].size = original_size - job->size;
        mm->partitions[partition_index + 1].is_free = 1;

        mm->partition_count++;
    }
}

```

The manual dynamic array insertion through shifting the partitions inserting new partitions and updating the metadata

MERGES AFTER DEALLOCATION

```
void merge_free_partitions(MemoryManager *mm) {
    int merged = 0;
    for (int i = 0; i < mm->partition_count - 1; i++) {
```

Call memorymanager again
Int i=0 mm starts at position zero
And it scans the partitions until we reach the second to the last section because it needs to look at the current section and the one next to it , if we went all the way to the end there would be nothing at the right to compare i++ to move to the next section

```
if (mm->partitions[i].is_free && mm->partitions[i + 1].is_free)
```

Determine whether the section is empty AND is the section to the right ALSO empty
If both yes merge the two empty sections

```
mm->partitions[i].size += mm->partitions[i + 1].size;
```

The actual merge operation makes the section bigger by adding the size of the second to it

```
1. for (int j = i + 1; j < mm->partition_count - 1; j++) { mm->partitions[j] = mm->partitions[j +
1]; }
2.
```

Cleanups
This for loop shuffles the partitions that were on the right of the merge to the left to close the gap left out by the merge operation

```
1. mm->partition_count--;
```

Update the partition count now the total no. of sections decrease by one

After the merge the new larger empty sec. might STILL be next to another empty section so we check AGAIN. One step back checking the same spot so we can merge three four or more adjacent empty sections in a row.

Compaction AGAIN

```
1. for (int i = 0; i < mm->partition_count; i++) {
2.     if (!mm->partitions[i].is_free) {
3.         temp_partitions[temp_count] = mm->partitions[i];
4.         temp_partitions[temp_count].start_address = current_addr;
5.         current_addr += temp_partitions[temp_count].size;
6.         temp_count++;
7.     }
8. }
9.
```

We have to collect round up all the partitions into a TEMPORARY partition away from the messy current partition that we have

```
1. f (!mm->partitions[i].is_free)
```

is this free? Checking whether the section is NOT free therefore !mm

```
1. temp_partitions[temp_count] = mm->partitions[i]
```

If there IS a partition, collect it and insert into the temp partition copy all its details and whatever job it belongs to.

```
1. temp_partitions[temp_count].start_address = current_addr
```

DECIDING ON THE PARTITIONS' NEW POSITION. The first partition will be put in the start of the mem. Address 0 then append to the partition in the temp partition

```
1. current_addr += temp_partitions[temp_count].size
```

Update plan for the job. If first partition is 10 units the next partition shall start from the 10 unit mark

UPDATE JOB ADDRESSES

```
 1. for (int i = 0; i < mm->job_count; i++) {
 2.     if (mm->jobs[i].is_allocated && !mm->jobs[i].is_completed) {
 3.         for (int j = 0; j < temp_count; j++) {
 4.             if (temp_partitions[j].job_id == mm->jobs[i].job_id) {
 5.                 mm->jobs[i].start_address = temp_partitions[j].start_address;
 6.                 break;
 7.             }
 8.         }
 9.     }
10. }
11.
```

Line1 ; go through ALL the jobs mm is tracking (mm stands for memorymanager)
Line2: focus only ON THE RUNNING jobs with memory allocated to them
For every active job we search thru the new temp partition layout
Line4 find the partition that belongs to the current job match the job id
Line5 before break update the job start address to the new compacted address calculated earlier

Add free MERGED partition at end

```
1. if (current_addr < mm->memory_size) {
2.     temp_partitions[temp_count].start_address = current_addr;
3.     temp_partitions[temp_count].size = mm->memory_size - current_addr;
4.     temp_partitions[temp_count].is_free = 1;
5.     temp_count++;
6. }
7.
```

Line1 check if there's still space and whether current addr = memsize cuz if it is true that means there's no more space to add
Line2 new free block starts where the last block ends
Line3 The size of the free block calculated , subtract total size of allocated block from total memory size
Line4 partition marked as free
Line5 new free partition added to temp_partition array to finish the new memory layout

```
1. for (int i = 0; i < temp_count; i++) {
2.     mm->partitions[i] = temp_partitions[i];
3. }
4. mm->partition_count = temp_count;
5.
```

Replace the old partition array now
Line1 for loop thru the new temp partition layout
Line2 copy from the temp array all the partitions back into the main mm primary partitions array replacing the old layout
Line3 total partition count in mem manager is updated (likely to be smaller than the original , separate free blocks were consolidated into one)